

Agentic AI Chapter 5: LangChain & LangGraph Stateful Workflow Mastery

Core Learning Objective: Master modern stateful agent systems using LangGraph. Understand cyclic graphs, typed state schemas, conditional branching edges, persistence checkpoints, and Human-in-the-Loop approval workflows.

1. Why LangGraph Replaced Linear Chains

Legacy frameworks (LangChain chains, LCEL) were designed as **Directed Acyclic Graphs (DAGs)**. But real agents are **inherently cyclic**: they try an action, inspect output, loop back, retry, ask clarifications, and refine results.

```
flowchart TD
    subgraph Legacy_Chain ["Legacy LangChain: Linear Chains (DAG)"]
        Input1["Prompt"] --> LLM1["LLM"]
        LLM1 --> Output1["Output"]
        Note1["Cannot loop! Hard to control branching state."]
    end

    subgraph LangGraph_Cyclic ["LangGraph: Stateful Cyclic Multi-Actor Graphs"]
        Start["__start__"] --> AgentNode["Agent Node (LLM Decides)"]
        AgentNode --> Branch{"Conditional Edge:<br/>Tool needed or Finished?"}
        Branch -->|Tool Call| ToolNode["Tool Node (Executes Python)"]
        ToolNode -->|Loops back!| AgentNode
        Branch -->|Done| EndNode["__end__"]
    end
```

2. Core LangGraph Primitives

1. **State:** A centralized typed dictionary (`TypedDict`) shared across all nodes in the graph. Nodes return partial updates that merge into the state.
2. **Nodes:** Plain Python functions that receive the current `State` , perform computation (call an LLM or execute a tool), and return updated keys.
3. **Edges:** Direct transitions from one node to another.

4. **Conditional Edges:** Dynamic routers that inspect the state and determine the next destination node.
-

3. Complete Production LangGraph Agent Implementation

```

from typing import TypedDict, Annotated, Sequence
import operator

# 1. Define the Global State Schema
class AgentState(TypedDict):
    messages: list[str]
    current_step: int
    tool_output: str | None
    is_finished: bool

# 2. Define Graph Nodes (Pure Python Functions)
def agent_node(state: AgentState) → dict:
    """Simulates LLM reasoning over the conversation history."""
    step = state.get("current_step", 0) + 1
    print(f"\n[Node: Agent] Evaluating step {step} ... ")

    if step == 1:
        # LLM decides it needs to query the database
        return {"current_step": step, "messages": state["messages"] + ["Calling QueryDB Tool"]}
    else:
        # LLM has tool output; concludes task
        return {"current_step": step, "is_finished": True, "messages": state["messages"]}

def tool_node(state: AgentState) → dict:
    """Executes external tool."""
    print("[Node: ToolExecutor] Running SQL query on warehouse ... ")
    result = "Total Sales: $1.4M across 4,200 orders"
    return {"tool_output": result, "messages": state["messages"] + [f"Tool Result: {result}"]}

# 3. Define Conditional Routing Function
def should_continue(state: AgentState) → str:
    if state.get("is_finished", False):
        return "end"
    if state.get("tool_output") is None:
        return "call_tool"
    return "agent"

# 4. In-Memory Graph Runner Emulating LangGraph Architecture
class CompiledGraph:

```

```
def __init__(self):
    self.nodes = {"agent": agent_node, "tool": tool_node}

def invoke(self, initial_state: AgentState) → AgentState:
    state = initial_state
    current_node = "agent"

    while True:
        # Execute active node
        updates = self.nodes[current_node](state)
        state.update(updates)

        # Evaluate conditional edge
        next_step = should_continue(state)
        if next_step == "end":
            print("\n[Graph] Reached __end__ state!")
            break
        elif next_step == "call_tool":
            current_node = "tool"
        else:
            current_node = "agent"

    return state

# --- Verification Driver ---
graph = CompiledGraph()
final_state = graph.invoke({
    "messages": ["User: What were our Q3 sales?"],
    "current_step": 0,
    "tool_output": None,
    "is_finished": False
})

print("\nFinal State Log:")
for msg in final_state["messages"]:
    print(f" * {msg}")
```

4. Human-in-the-Loop & Time-Travel Checkpointing

In production, agents executing destructive actions (e.g. modifying financial ledgers, sending mass emails, dropping tables) must be paused for **Human Approval**:

```
sequenceDiagram
    autonumber
    participant Graph as LangGraph Engine
    participant Checkpoint as Postgres Checkpointer
    actor Human as Human Reviewer

    Graph->>Graph: Agent plans: delete_old_customer_records()
    Note over Graph: Interrupt Flag Hit (Before destructive tool)!
    Graph->>Checkpoint: Persist exact state snapshot to Postgres
    Graph->>Human: Alert: "Agent wants to delete 1,000 records. Approve? (Y/N)"

    alt Human Approves
        Human->>Graph: POST /resume {action: "approve"}
        Graph->>Checkpoint: Reload snapshot
        Graph->>Graph: Executes tool and continues graph!
    else Human Rejects
        Human->>Graph: POST /resume {action: "reject", feedback: "Only archive them"}
        Graph->>Graph: Agent re-plans with human guidance
    end
end
```

Time-travel checkpointing allows developers to inspect every historical node execution, modify past state variables, and branch alternative execution paths during debugging.